MLOps Lab Assignment 2

Tracking a Pre-trained Sentiment Model with MLflow

Name	Prarthana N
USN	4GW23CI039
Experiment	sentiment-inference-lab
Model	distilbert-base-uncased-finetuned-sst-2-english (pre-trained, inference only)
Tools	Python, MLflow, Hugging Face Transformers, PyTorch, pandas
Versions / Platform	Python 3.13.2, MLflow 3.17.2, transformers 5.21.0, torch 2.15.0, pandas 3.1.0; Windows 11, Command Prompt, virtual environment mlops-env

1. Objective

This lab uses MLflow to record an inference-only experiment. A ready-made Hugging Face sentiment classifier (DistilBERT fine-tuned on SST-2) labels ten short sentences, five positive and five negative. For every run, MLflow stores the parameters used, the metrics obtained and a *predictions.csv* artifact. The same model is run with different batch sizes (1, 8 and then 4) and the runs are compared in the MLflow UI to see how batch size affects speed and accuracy. No training is performed.

2. Environment Setup

The steps from the assignment sheet were followed in the terminal, inside a virtual environment named *mlops-env*.

Step	Command	Purpose
1	python --version pip --version	Confirm Python (3.9 to 3.12) and pip are installed
2	python -m venv mlops-env mlops-env\Scripts\activate	Create and activate an isolated environment
3	python -m pip install --upgrade pip	Upgrade pip
4	pip install mlflow transformers torch pandas	Install tracking, model, engine and CSV libraries
5	python -c "import mlflow, transformers, torch, pandas; print(mlflow.__version__)"	Verify the installation
6	pipeline('sentiment-analysis', model=...)	Pre-download the ~250 MB model once; later runs use the cache

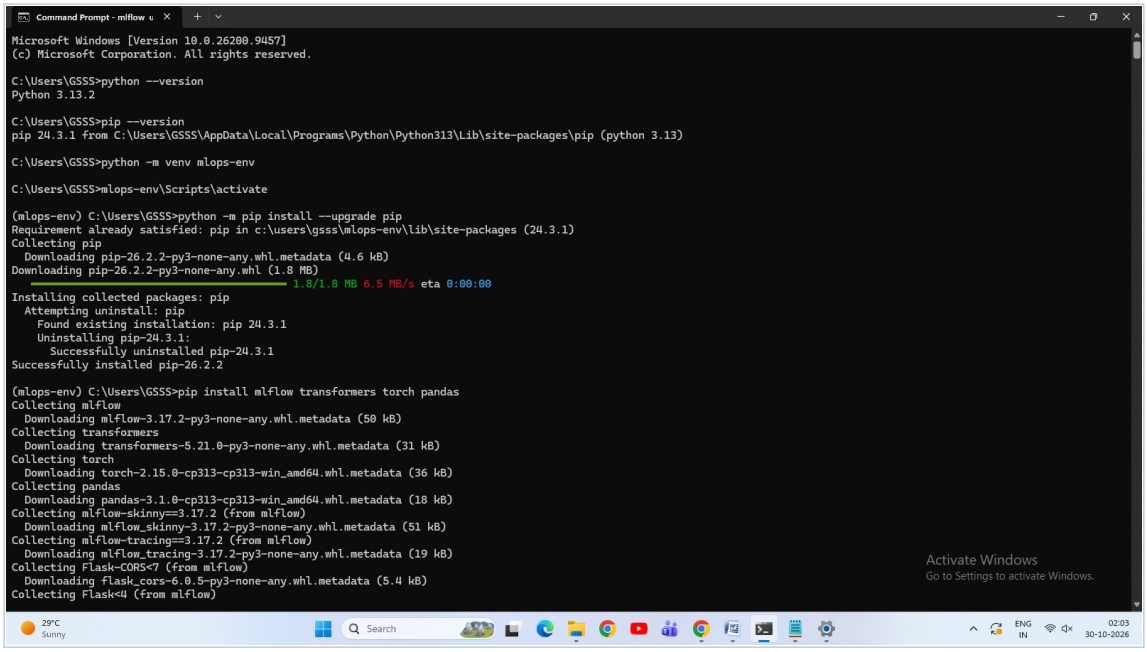


Figure 1: Version checks, virtual environment, pip upgrade and package installation

3. Task 1: Tracked Experiment

The script *mlflow_lab.py* builds a Hugging Face *pipeline* for the model and passes the ten sentences through it with a chosen *batch_size*. Inside *mlflow.start_run()* it logs three **parameters** (model_name, batch_size, num_samples), three **metrics** (accuracy, avg_confidence, total_latency_sec) and one **artifact** (predictions.csv holding text, actual label, predicted label and confidence). Inference time is measured with *time.time()*. The complete script, including the third run added for Task 3, is below.

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE"),
]
texts = [t for t, _ in data]
labels = [l for _, l in data]

mlflow.set_experiment("sentiment-inference-lab")

def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)
    with mlflow.start_run(run_name=run_name):
        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
```

```
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]
        accuracy = sum(p == l for p, l in zip(pred_labels, labels)) / len(labels)
        avg_conf = sum(p["score"] for p in preds) / len(preds)

        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)

        # Artifact
        df = pd.DataFrame({
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds],
        })
        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))

MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")   # Task 3
```

The script is executed with `python mlflow_lab.py`; each run prints its accuracy and latency. Then `mlflow ui` starts the tracking server at http://127.0.0.1:5000.

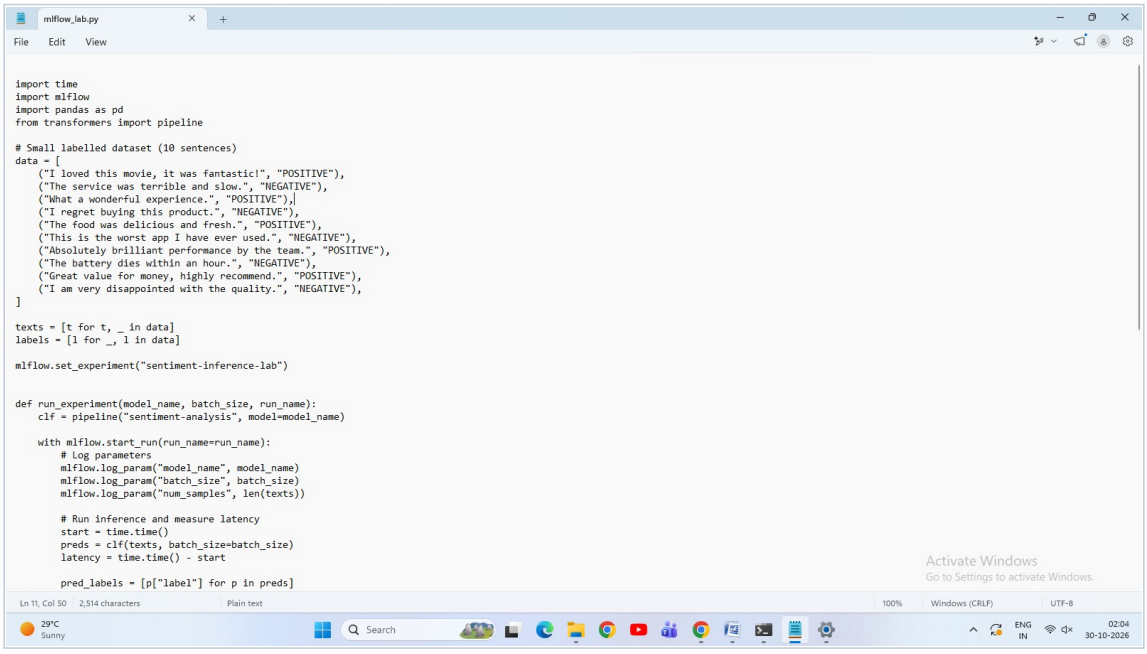


Figure 2: mlflow_lab.py open in Notepad

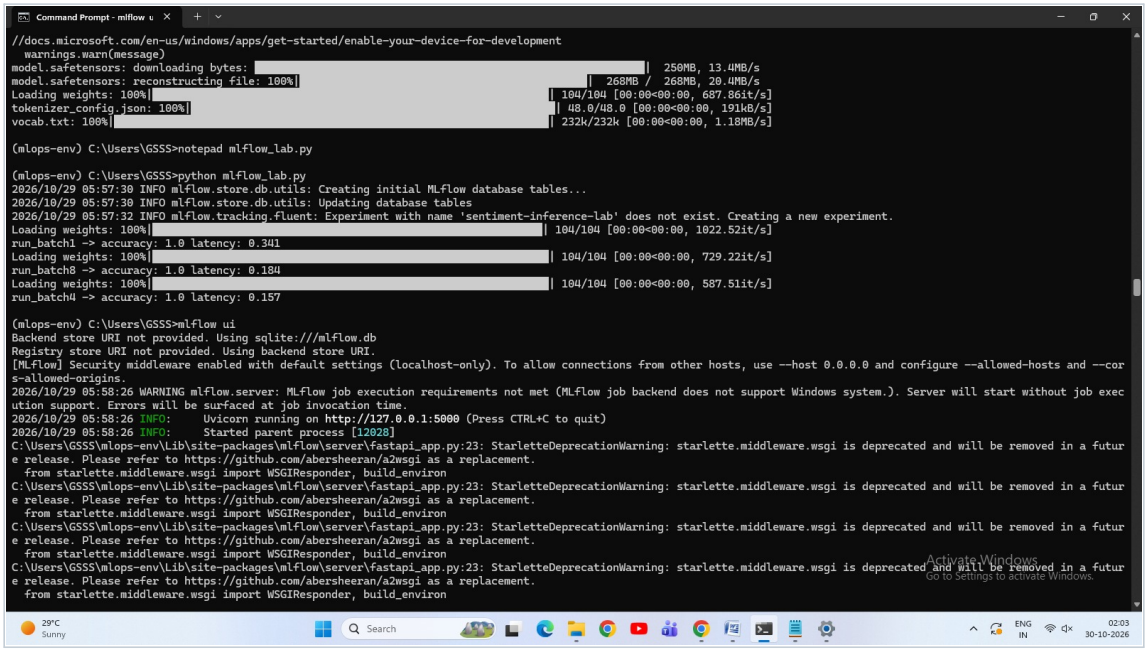


Figure 3: Terminal output of the three runs and start-up of the MLflow UI server

Results of the runs (read from the MLflow UI):

Run	batch_size	accuracy	avg_confidence	total_latency_sec
run_batch1	1	1.0	0.9997	0.341
run_batch8	8	1.0	0.9997	0.184
run_batch4	4	1.0	0.9997	0.157

4. Task 2: Exploring the MLflow UI

With `mlflow ui` running, http://127.0.0.1:5000 was opened and the experiment *sentiment-inference-lab* selected. The runs table lists each run with its parameters and metrics. The runs were then ticked and opened with **Compare**, which shows batch_size against accuracy and latency side by side. Finally one run was opened to view *predictions.csv* under the Artifacts tab.

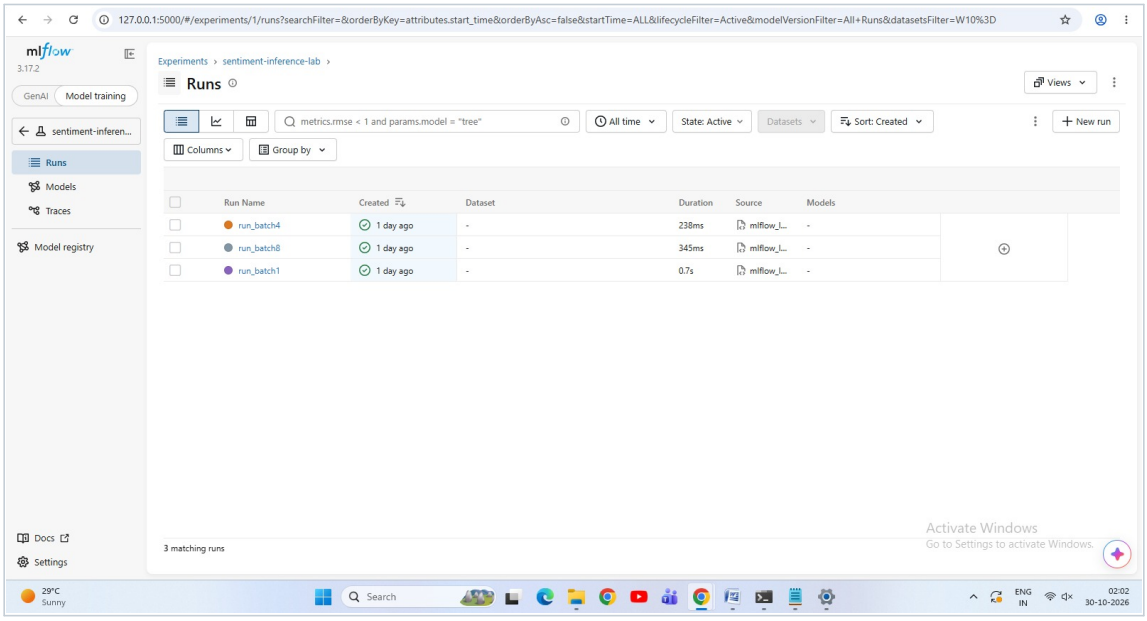


Figure 4: Runs table of sentiment-inference-lab showing the 3 runs

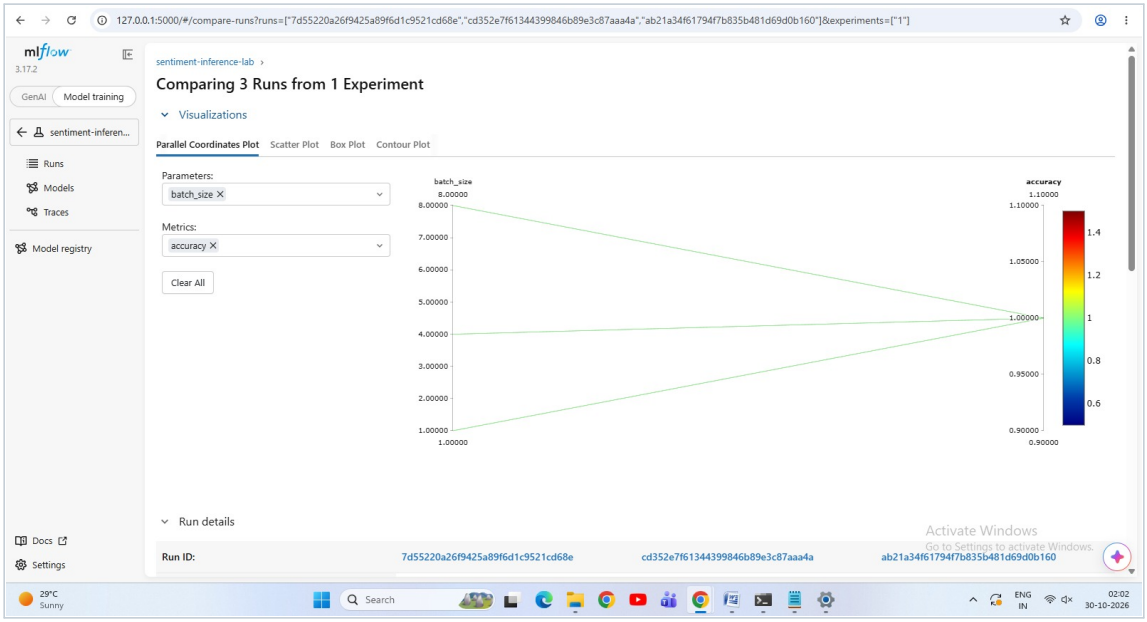


Figure 5: Compare view of the three runs: batch_size against accuracy

The individual run pages show the logged parameters and metrics. Pages of run_batch1 and run_batch8 are shown below; each run's predictions.csv is stored under its Artifacts tab.

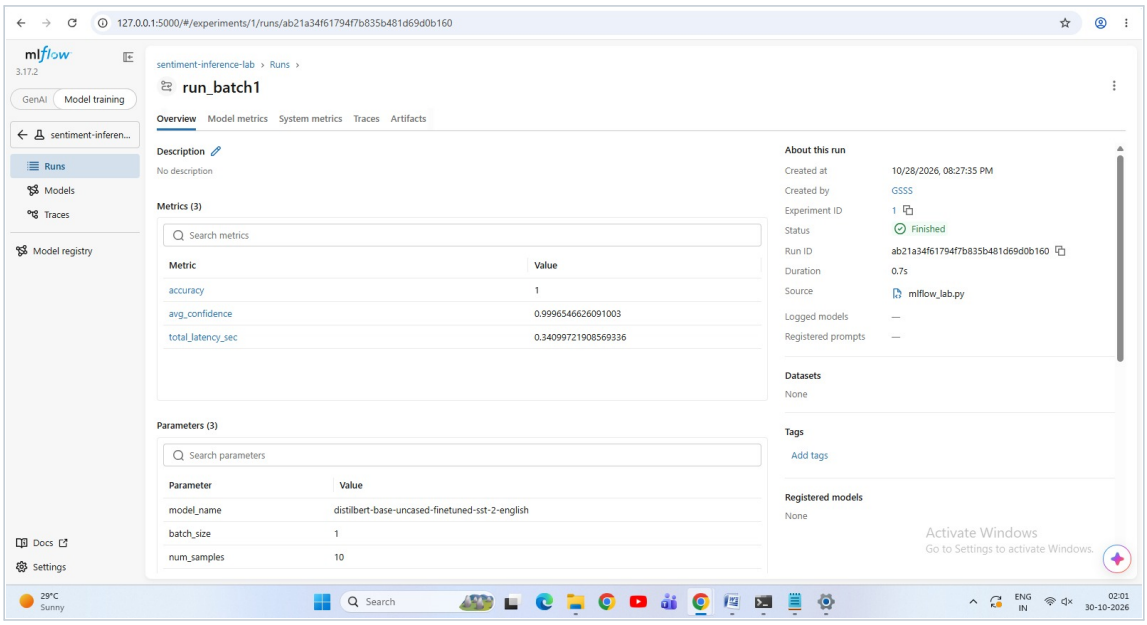


Figure 6: run_batch1 overview: metrics and parameters (batch_size = 1)

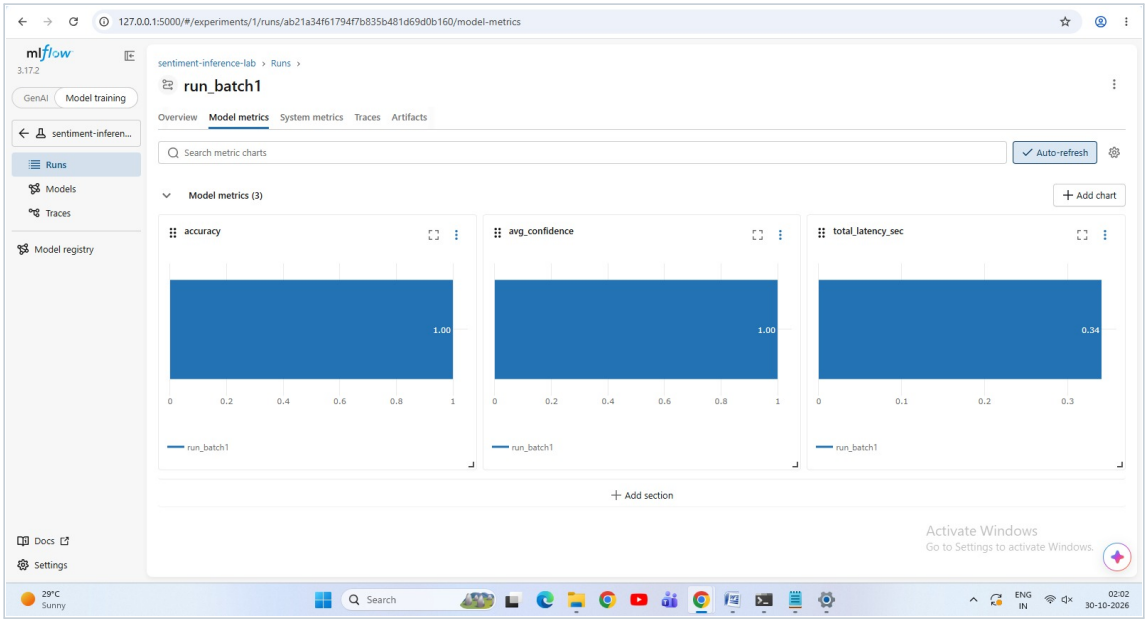


Figure 7: run_batch1 model metrics: accuracy, avg_confidence, total_latency_sec

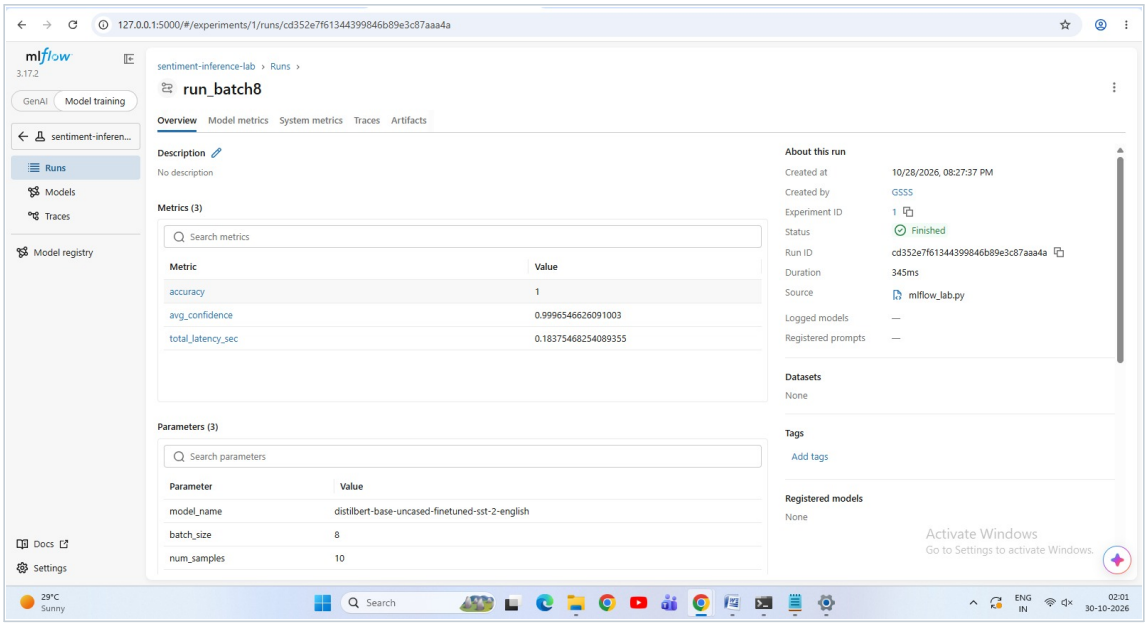


Figure 8: run_batch8 overview: metrics and parameters (batch_size = 8)

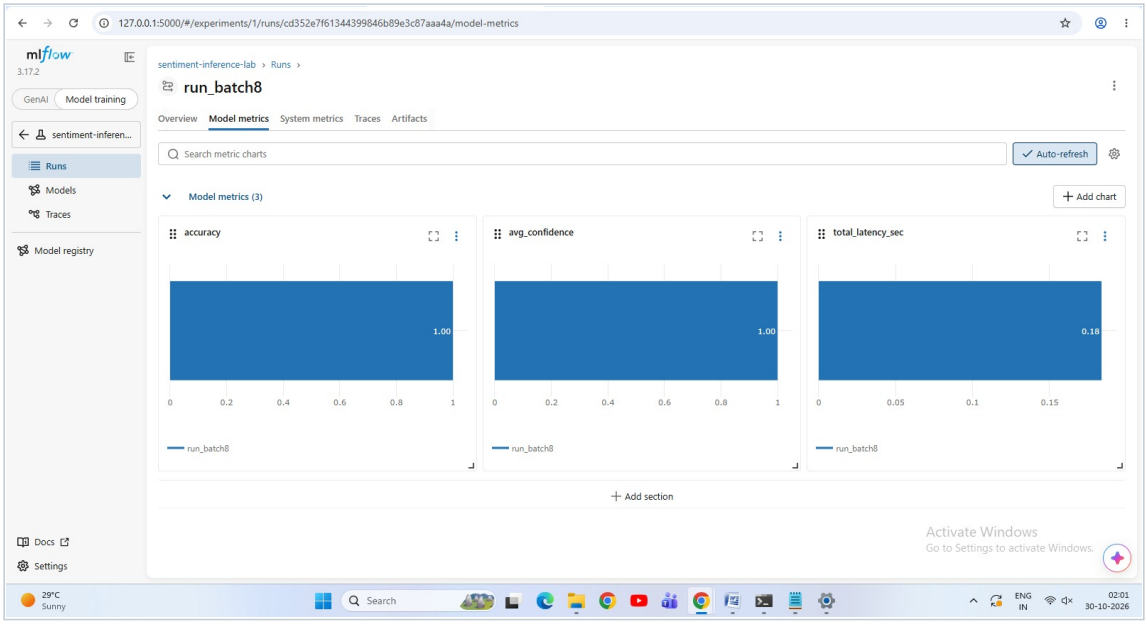


Figure 9: run_batch8 model metrics

5. Task 3: Third Experiment

One more call was added at the bottom of the script, using the same model with **batch_size=4** and the run name *run_batch4*:

```
run_experiment(MODEL, batch_size=4, run_name="run_batch4")   # Task 3
```

After running the script again, the new run appeared in the MLflow UI as the top row of the runs table (Figure 4), confirming that it was logged correctly.

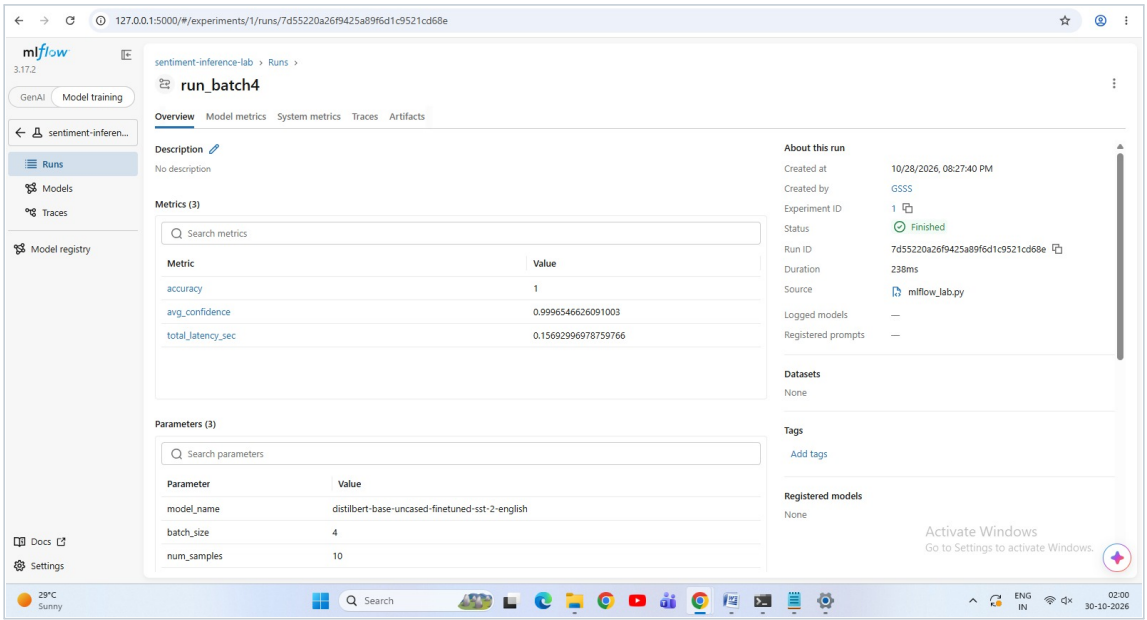


Figure 10: run_batch4 overview: metrics and parameters (batch_size = 4)

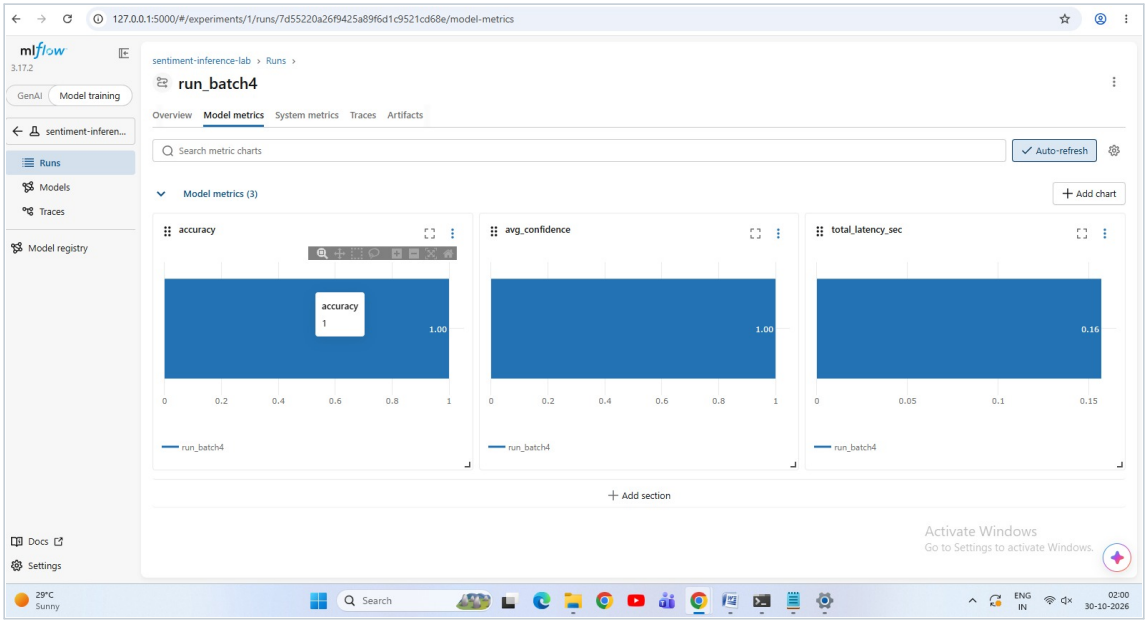


Figure 11: run_batch4 model metrics

6. Task 4: Short Report

1. Lowest latency: run_batch4 had the lowest latency (0.157 s), followed by run_batch8 (0.184 s) and run_batch1 (0.341 s). Batching lets the model process several sentences in one forward pass, so less time is spent on repeated per-call overhead; batch size 1 needs ten separate passes, and the first run may also include warm-up cost. With only ten sentences, small differences can be normal run-to-run variation.

2. Accuracy: It did not change between runs. The model, its weights and the ten sentences were identical every time; batch size only changes how inputs are grouped for computation, not what the model predicts.

3. Benefits of MLflow: (i) Parameters, metrics and the predictions file are saved against each run ID, so any run can be reopened and verified later. (ii) The UI compares runs side by side with tables and charts, which is easier than reading printed terminal lines.

7. Submission Checklist

Item required	Where it is in this report
Screenshot of the MLflow runs table	Section 4, Figure 4
Screenshot of the Compare view	Section 4, Figure 5
mlflow_lab.py file	Section 3 (full code and Notepad screenshot)
Answers to the Task 4 questions	Section 6